



INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



**Advanced Visual Programming
Lab**

Assignment No 01

Submitted To:

Sir Uzair

Submitted By:

Momina Waqar

232201039

BSCS 6 A

BS in Computer Science,

K.I.C.S.I.T , Kahuta

31-March-2026

1 Introduction to AI-Based Applications

1.1 What is Artificial Intelligence?

Artificial Intelligence:

Artificial Intelligence (AI) is the simulation of human intelligence in machines that can think, learn, and make decisions.

1.2 What are AI-Based Applications?

AI-Based Applications:

AI-based applications are advanced software systems that use Artificial Intelligence techniques to perform tasks that normally require human intelligence. These applications can learn from data, improve their performance over time, and make decisions with minimal human intervention. They use technologies such as:

- Machine Learning (ML) – for learning patterns from data
- Natural Language Processing (NLP) – for understanding human language
- Computer Vision – for analyzing images and videos

1.3 Q3: Real-world examples

Some common real-world examples of AI-based applications are:

- **Chatbots:** These are AI programs that can interact with users through text or voice. They are commonly used in customer support to answer queries automatically.
- **Recommendation Systems:** These systems suggest products, movies, or content based on user preferences. For example, platforms like Netflix and Amazon use AI to recommend items.
- **Image Recognition Systems:** These applications can identify objects, faces, or patterns in images. They are used in security systems, medical diagnosis, and social media platforms.

2 AI in .NET Ecosystem

2.1 How AI is integrated in .NET

AI can be integrated in .NET applications using machine learning frameworks, APIs, and cloud services. In .NET, developers can either:

- Build their own machine learning models using ML.NET
- Use pre-trained AI models through REST APIs
- Integrate cloud AI services like Azure AI

2.2 Introduction to ML.NET

ML.NET is a free, open-source, and cross-platform machine learning framework developed by Microsoft for .NET developers. It allows developers to build, train, and deploy machine learning models using C# without requiring deep knowledge of data science.

Key features of ML.NET include:

- Model training and evaluation
- Data preprocessing tools
- Support for tasks like classification, regression, and clustering
- Integration with existing .NET applications

ML.NET is especially useful for building custom AI solutions such as image classification, sentiment analysis, and prediction systems.

2.3 REST APIs for AI services (e.g., OpenAI APIs)

REST APIs allow .NET applications to communicate with external AI services over the internet. Instead of building models from scratch, developers can use powerful pre-trained AI models provided by services like OpenAI.

In this approach:

- The application sends a request (input data) to the API
- The AI service processes the request
- A response (output) is returned to the application

2.4 Benefits of using .NET for AI applications

Using .NET for AI-based applications provides several advantages:

- **Strong ecosystem:** Rich libraries and tools provided by Microsoft
- **Cross-platform support:** Runs on Windows, Linux, and macOS
- **Easy integration:** Seamless connection with APIs and databases
- **High performance:** Efficient and scalable applications
- **Developer-friendly:** Uses C#, which is easy to learn and widely used

3 AI Application Development Lifecycle

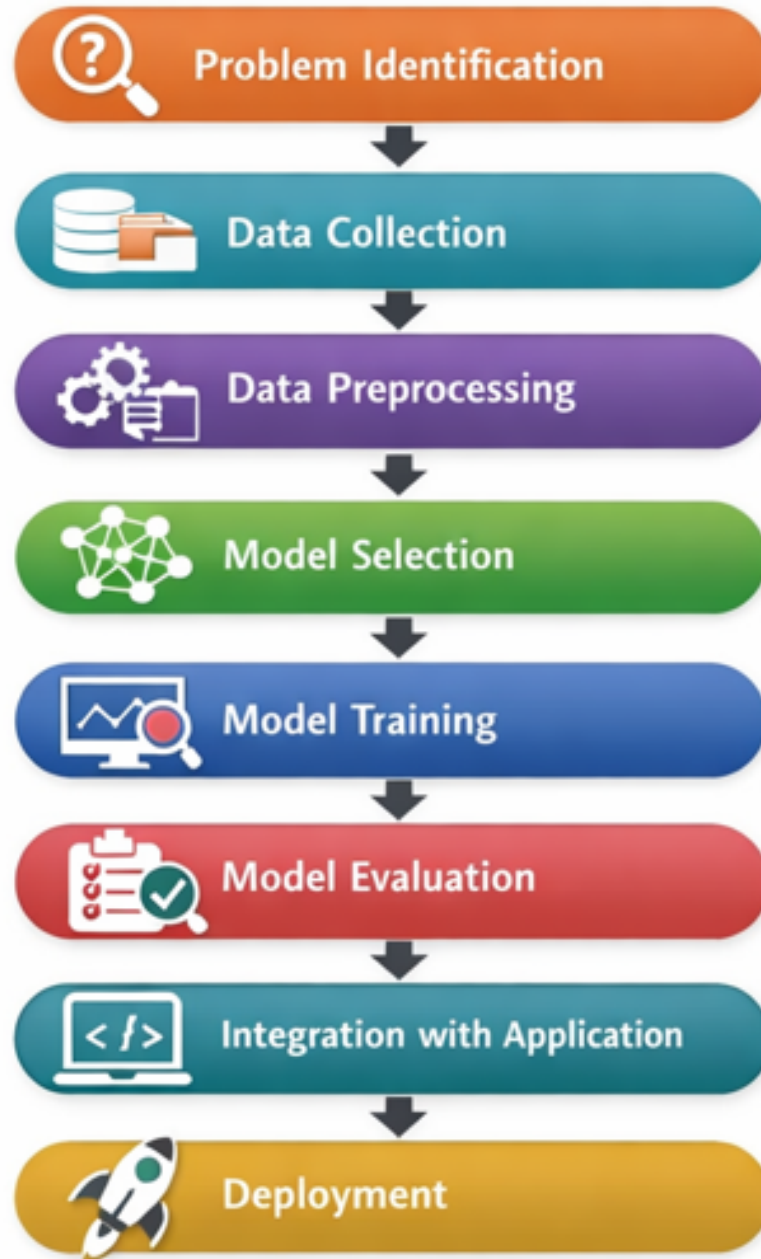
3.1 Explain the complete process

The development of an AI-based application follows a structured lifecycle. The main steps are:

1. **Problem Identification:** Define the problem clearly and understand the goal of the application.
2. **Data Collection:** Gather relevant data required to train the AI model.
3. **Data Preprocessing:** Clean and prepare the data by removing errors, handling missing values, and formatting it properly.
4. **Model Selection:** Choose the appropriate machine learning model based on the problem type.
5. **Model Training:** Train the model using the prepared data so it can learn patterns.
6. **Model Evaluation:** Test the model to check its accuracy and performance.
7. **Integration with Application:** Connect the trained model with the .NET application.
8. **Deployment:** Deploy the application so it can be used by users.

Diagram:

AI Application Development Lifecycle



4 Architecture of an AI-Based .NET Application

The architecture of an AI-based .NET application consists of four main components and the data flow between them:

1. Frontend (UI – Windows Forms / Web App)

This is the interface visible to the user. Users can upload images or provide other inputs through the UI.

- Example: In a Windows Forms app, buttons, image viewer, and labels are used; in a web app, HTML forms and buttons are used.
- The frontend sends user input to the backend and displays the AI prediction results.

2. Backend (.NET Logic)

The backend contains C# code that handles the application logic. Its responsibilities include:

- Validating user input
- Communicating with the AI model or API
- Fetching/storing data from/to the database
- Processing the output and sending it back to the frontend

3. AI Model / API (ML.NET or REST API)

This component provides intelligence to the application.

- For image classification, ML.NET can use a pre-trained or custom-trained model.
- If using a REST API, the backend sends input to the API and receives the predicted output.
- The data flow inside this component is: *input* $\rightarrow$ *processing* $\rightarrow$ *prediction*.

4. Database (Optional)

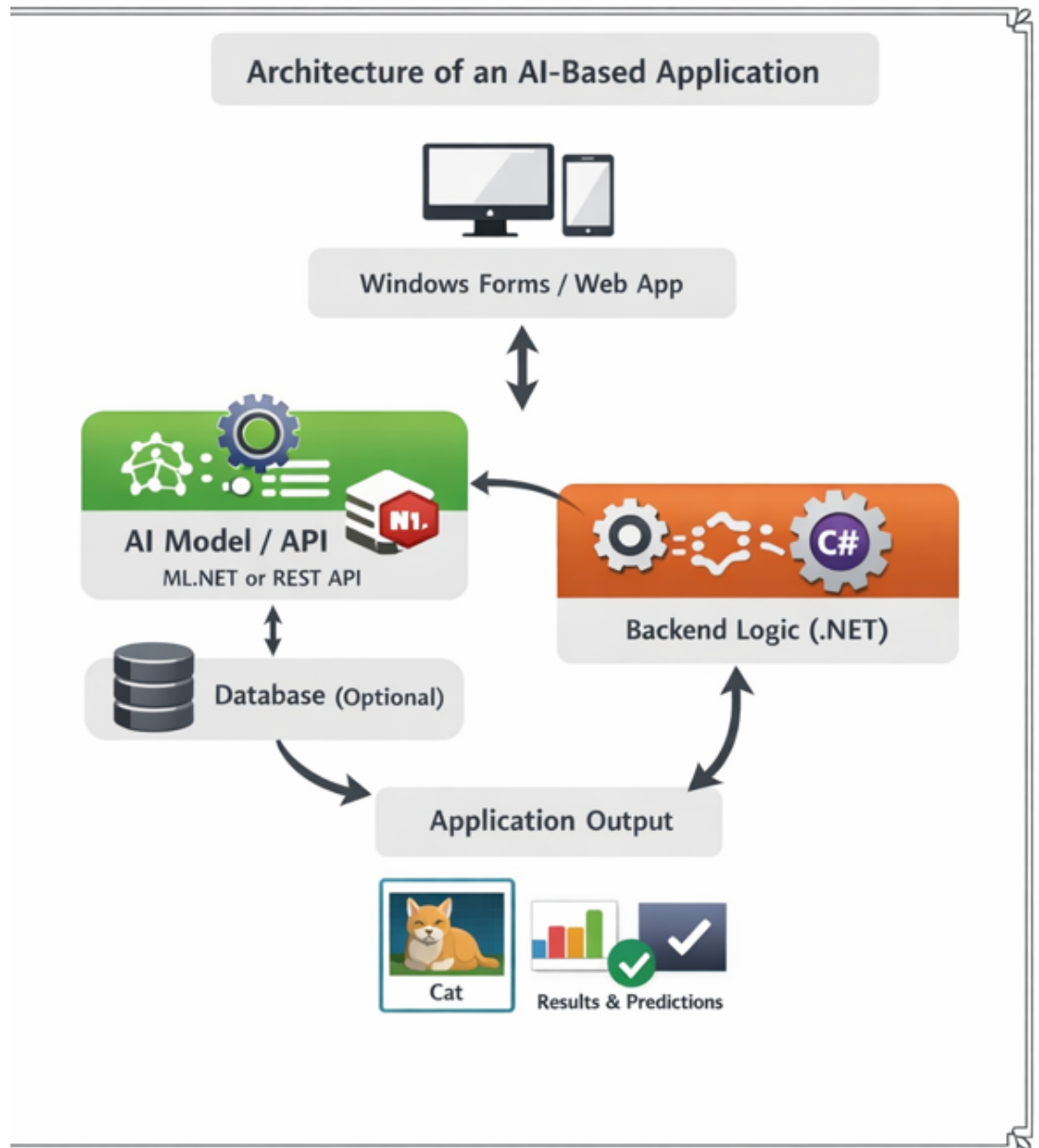
If required, input images, prediction results, user data, or logs can be stored in a database.

- Example: SQL Server, SQLite, or MongoDB

Application Output:

After processing, the AI model prediction is displayed on the frontend. Example: the uploaded image is classified as “Cat”.

Diagram:



5 API Integration Concept

5.1 What is an API?

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate and exchange data with each other. It acts as an intermediary between the client (application) and the service (server or external system).

5.2 How AI APIs are used in applications

AI APIs enable applications to leverage pre-trained AI models without building models from scratch. The application sends input data to the AI API, which processes the data using its machine learning or deep learning models and returns the output.

Examples:

- Sending text to a sentiment analysis API and receiving the sentiment score.
- Sending an image to an image classification API and receiving the predicted label.
- Sending a question to a chatbot API and receiving a generated response.

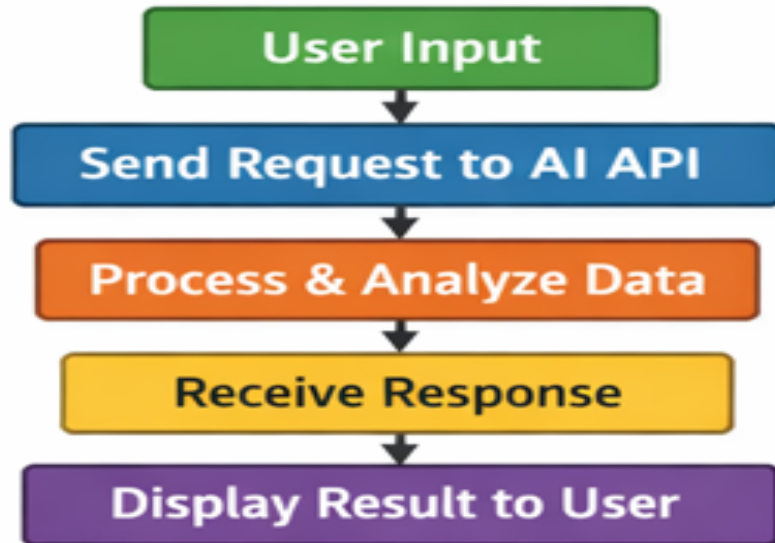
5.3 Request/Response Flow

The typical flow for using an AI API is as follows:

User Input → Application → AI API → Application → User Output

- **Request:** The application sends user input (text, image, or other data) to the AI API.
- **Processing:** The AI API processes the input using its model or algorithm.
- **Response:** The AI API returns the processed output back to the application.
- **Display:** The application shows the AI-generated result to the user.

AI Request/Response Flow



6 Introduction to Docker

6.1 What is Docker?

Docker is an open-source platform that allows developers to package applications and their dependencies into lightweight, portable containers. These containers can run consistently across different environments, ensuring that the application behaves the same on development, testing, and production systems.

6.2 Why Docker is used

Docker is used to solve the common problem of "it works on my machine" by isolating the application from the underlying operating system and dependencies. It simplifies deployment, scaling, and maintenance of applications.

6.3 Benefits of Docker

- **Portability:** Containers can run on any system that has Docker installed, without worrying about environment differences.

- **Consistency:** Ensures the application behaves the same across development, testing, and production.
- **Efficiency:** Containers are lightweight and use system resources efficiently.
- **Scalability:** Makes it easy to scale applications horizontally by running multiple container instances.

7 Version Control using GitHub

7.1 Importance of GitHub

GitHub is a cloud-based platform that provides version control using Git. It allows developers to:

- Track changes in the source code over time
- Collaborate with team members on the same project
- Maintain backup copies of projects
- Manage project documentation and issues

7.2 Basic Workflow

The basic workflow for using GitHub in a project includes:

1. **Repository Creation:** Create a new repository on GitHub to store your project code and files.
2. **Commit & Push:** Make changes locally, commit them with meaningful messages, and push the changes to the GitHub repository.
3. **README File:** Include a README.md file in the repository to provide a project overview, setup instructions, and any important notes.

Work FLOW:



8 Ethical Considerations

AI applications have the potential to impact society significantly. It is important to consider the following ethical aspects:

8.1 Bias in AI

AI models can inherit biases from the training data, leading to unfair or discriminatory outcomes. Developers must ensure training data is diverse and representative, and evaluate models for potential bias.

8.2 Data Privacy

AI applications often use personal or sensitive data. Protecting user privacy is crucial. Proper data handling, anonymization, and secure storage practices should be followed.

8.3 Responsible AI Usage

AI should be used responsibly to avoid misuse. Developers must ensure their applications do not harm individuals or communities, and should provide transparency and accountability in AI decision-making.

THE END